S09 · Que el aviso no tumbe la transferencia

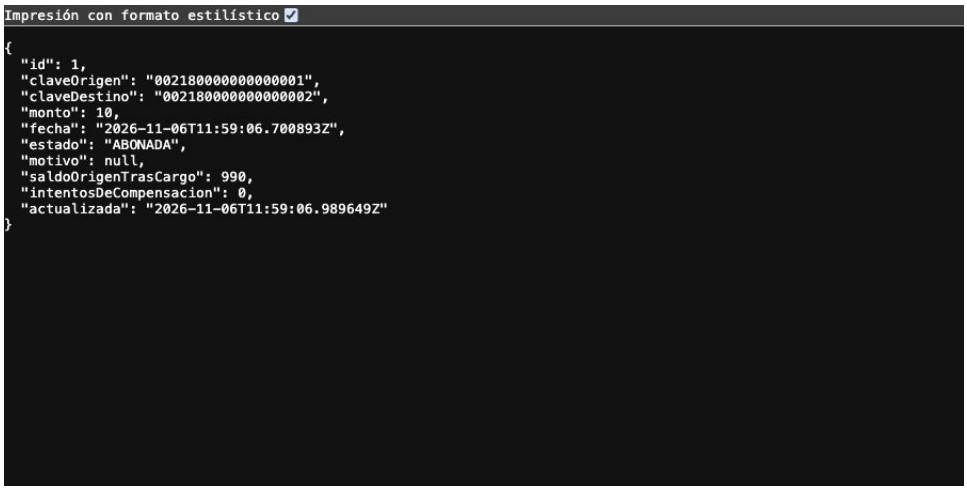
Pablo Vaquero · Cuenta 318259624 · Grupo 2
Sistemas Distribuidos · 6 de noviembre de 2026

RabbitMQ separa el movimiento de dinero de la entrega del aviso. La prueba incluye consumidor apagado, mensajes duplicados, una devolución y un mensaje inválido. Es una recuperación fuera del plazo original; no se presupone su registro en el corte.

Parte 0 · El problema de la llamada síncrona

Observación	Resultado real
POST transferencia de $10	500
Estado guardado	ABONADA
Saldo de Ana / Beto	990.00 / 510.00

Se apagó Notificación sobre la base oficial sin modificar. El cargo y el abono ya estaban hechos, pero la llamada HTTP del aviso falló antes de marcar COMPLETADA: la respuesta fue 500 y el estado quedó ABONADA. Un fallo del aviso se presentó como fallo de la transferencia.



Captura real del GET /api/transferencias/1: estado ABONADA. Respuesta del POST y saldos completos: 01-problema-base.json.

Base: 05-mensajeria-base, commit 22c52075f70909182748f28bf9c487632144796b. Proyecto Docker mexibanco-s09-pablo; gateway 18082. Los horarios con Z están en UTC; son pruebas locales con datos ficticios del enunciado.

Parte 1 · Publicar un hecho

La saga guarda COMPLETADA o COMPENSADA y después publica TransferenciaRealizada o TransferenciaRevertida. Se retiró la dependencia HTTP de Notificación de este flujo. EventoTransferencia conserva exactamente los campos del contrato y deriva un UUID estable de transferenciaId y tipo.

```
transferencia.completar();
transferencias.save(transferencia);
publicador.publicar(EventoTransferencia.realizada(...));

// RabbitTemplate: JSON + correlated confirms + mandatory
// deliveryMode=PERSISTENT; messageId=eventoId
```

Pieza	Contrato
Exchange	transferencias · topic · durable
Claves	transferencia.realizada / transferencia.revertida
Cola y binding	notificacion.transferencias · transferencia.*
DLX / DLQ	transferencias.dlx / notificacion.transferencias.dlq

El publisher confirm acredita la aceptación del broker, no la ejecución del consumidor. Si ninguna cola coincide, el broker puede confirmar sin guardar el mensaje en una cola. mandatory permite recibir la devolución y tratarla como fallo. Durabilidad de cola y persistencia del mensaje permiten sobrevivir al reinicio del nodo, dentro de sus límites operativos.



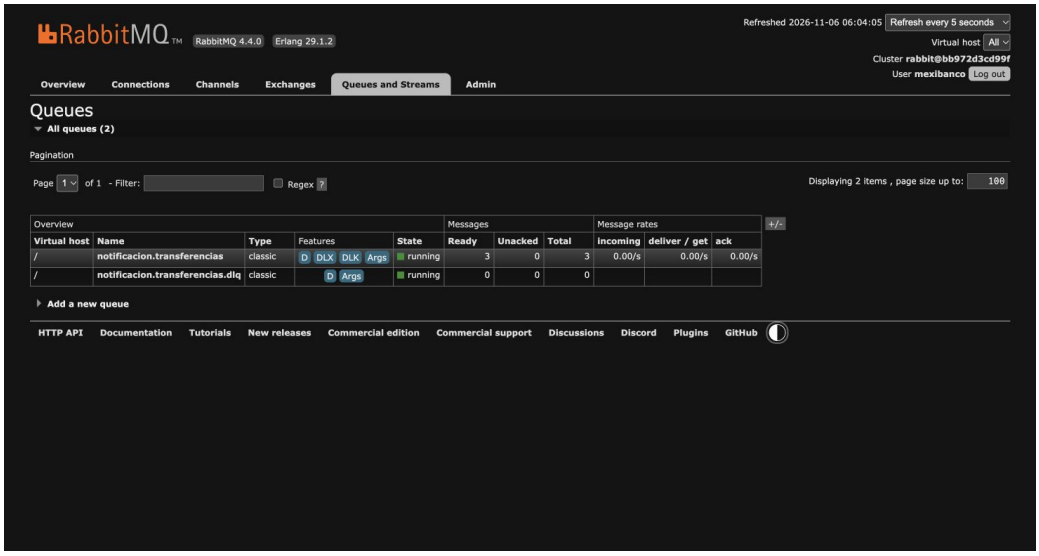
Captura real del exchange en RabbitMQ. La topología se declara con beans; el productor espera la confirmación correlacionada.

Se retiró temporalmente solo el binding, conservando las colas. La transferencia 11 quedó COMPLETADA pero respondió 500: mandatory detectó el mensaje sin ruta. Se restauró el binding en finally. A diferencia de un productor sin mandatory, esta versión hace visible esa pérdida potencial; el estado y la publicación todavía no son atómicos.

Parte 2 · Desacople en tiempo

Transferencia	HTTP	Estado
5	201	COMPLETADA
6	201	COMPLETADA
7	201	COMPLETADA

Con Notificación apagada, las tres transferencias de $1 respondieron 201. La cola ya existía y guardó tres mensajes Ready, sin consumidores. Al volver el consumidor, pasó a cero y las dos bandejas respondieron 200 con sus avisos. El dinero y el aviso pueden progresar en momentos distintos.



Captura real durante la pausa: tres mensajes Ready. Evidencias HTTP y estadísticas: 02-tres-ready.json y 03-consumidos.json.

El listener usa ack manual. Invoca un método @Transactional que escribe dos avisos por realizada, o uno por revertida; solo al regresar ese método tras el commit se hace basicAck. Si falla PostgreSQL, basicNack con requeue=true conserva el trabajo para otra entrega.

Impresión con formato estilístico

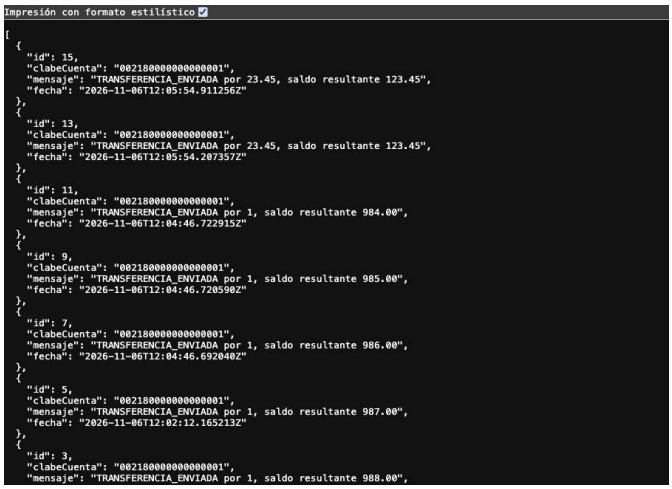
```
[
  {
    "id": 22,
    "clabeCuenta": "00218000000000001",
    "mensaje": "TRANSFERENCIA_ENVIADA por 3, saldo resultante 979.00",
    "fecha": "2026-11-06T12:17:50.757740Z"
  },
  {
    "id": 20,
    "clabeCuenta": "00218000000000001",
    "mensaje": "TRANSFERENCIA_ENVIADA por 2, saldo resultante 982.00",
    "fecha": "2026-11-06T12:17:08.132873Z"
  },
  {
    "id": 19,
    "clabeCuenta": "00218000000000001",
    "mensaje": "DEVOLUCION_TRANSFERENCIA por 5, saldo resultante 984.00",
    "fecha": "2026-11-06T12:16:50.147655Z"
  },
  {
    "id": 17,
    "clabeCuenta": "00218000000000001",
    "mensaje": "TRANSFERENCIA_ENVIADA por 27.89, saldo resultante 123.45",
    "fecha": "2026-11-06T12:07:20.870915Z"
  },
  {
    "id": 15,
    "clabeCuenta": "00218000000000001",
    "mensaje": "TRANSFERENCIA_ENVIADA por 23.45, saldo resultante 123.45",
    "fecha": "2026-11-06T12:05:54.911256Z"
  },
  {
    "id": 13,
    "clabeCuenta": "00218000000000001",
    "mensaje": "TRANSFERENCIA_ENVIADA por 23.45, saldo resultante 123.45",
    "fecha": "2026-11-06T12:05:54.207357Z"
  },
  {
    "id": 11,
    "clabeCuenta": "00218000000000001",
    "mensaje": "TRANSFERENCIA_ENVIADA por 1, saldo resultante 984.00",
    "fecha": "2026-11-06T12:04:46.722915Z"
  },
  {
```

Bandeja real al cierre: avisos normales y DEVOLUCION_TRANSFERENCIA por $5. La transferencia 8 fue COMPENSADA y dejó un aviso de devolución; 08-compensada.json conserva la respuesta y la bandeja.

Parte 3 · Duplicado: antes y después

Se publicó dos veces el mismo JSON y eventoId usando la API de administración de RabbitMQ. El monto distingue cada prueba. Son eventos sintéticos con transferenciaId 99999: prueban notificaciones, no representan cargos reales.

Modo	Monto sintético	Tras publicar 1 / 2
SIN_IDEMPOTENCIA=true	23.45	[2, 4]
Protección activa	27.89	[2, 2]



Antes: dos avisos de envío idénticos en la bandeja. Por cada publicación también se insertó el aviso de recepción: cuatro registros SQL en total.

Parte 3 · Un solo efecto por evento

El arreglo inserta eventoId como llave primaria con INSERT ... ON CONFLICT DO NOTHING, en la misma transacción que los avisos. Si devuelve cero, ya se procesó: no se crean avisos y se confirma. Si guardar un aviso falla, también se revierte la marca. La restricción resuelve carreras entre dos entregas; consultar y luego insertar sin restricción no bastaría.

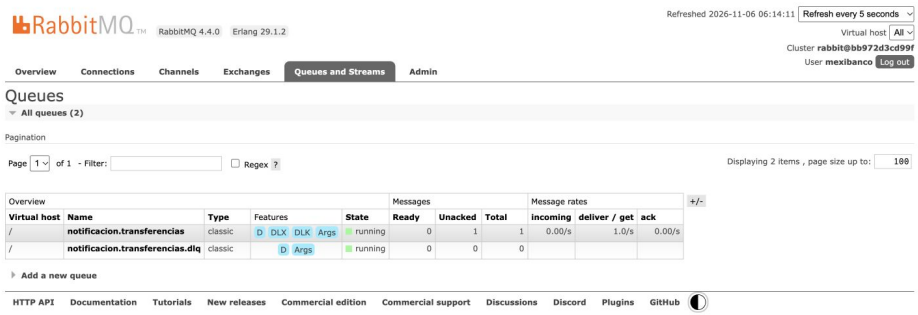

```
[
  {
    "id": 22,
    "clabeCuenta": "002180000000000001",
    "mensaje": "TRANSFERENCIA_ENVIADA por 3, saldo resultante 979.00",
    "fecha": "2026-11-06T12:17:50.757740Z"
  },
  {
    "id": 20,
    "clabeCuenta": "002180000000000001",
    "mensaje": "TRANSFERENCIA_ENVIADA por 2, saldo resultante 982.00",
    "fecha": "2026-11-06T12:17:08.132873Z"
  },
  {
    "id": 19,
    "clabeCuenta": "002180000000000001",
    "mensaje": "DEVOLUCION_TRANSFERENCIA por 5, saldo resultante 984.00",
    "fecha": "2026-11-06T12:16:50.147655Z"
  },
  {
    "id": 17,
    "clabeCuenta": "002180000000000001",
    "mensaje": "TRANSFERENCIA_ENVIADA por 27.89, saldo resultante 123.45",
Impresión con formato estilístico ☑
  },
  {
```


Bandeja real al cierre: el envío sintético de 27.89 aparece una sola vez. El SQL de 05-duplicado-resuelto.json verifica los dos destinatarios: 2/2 tras repetir.

El JSON del modo ingenuo conserva un GET 503 durante el refresco del directorio; los conteos SQL 2/4 y los logs sí se obtuvieron. La captura se hizo después de recuperar la bandeja. No se reemplazó ese error por un resultado inventado.

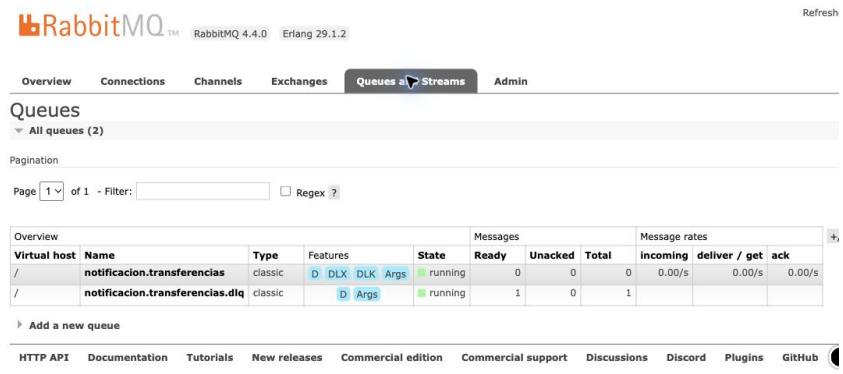
Parte 4 · El mensaje que nunca será válido

Se publicó exactamente un cuerpo incompleto: {"hola": . En el modo defectuoso REENCOLAR_INVALIDOS=true, el consumidor lo rechazó y volvió a ponerlo en la misma cola; se acumularon reentregas. La DLQ ya estaba declarada, pero no servía mientras el consumidor pedía requeue=true.



Antes: mensaje en la cola principal y DLQ vacía. 06-veneno-antes.json conserva estadísticas y los errores de parseo; no se publicó una segunda copia.

Se restauró el modo final, REENCOLAR_INVALIDOS=false. Los errores permanentes de formato o contrato hacen basicNack(tag, false, false). Los argumentos x-dead-letter-exchange y x-dead-letter-routing-key llevan el mensaje a transferencias.dlx, y de ahí a la DLQ. Las fallas temporales de almacenamiento siguen reencolándose.



Después: principal 0, DLQ 1. La transferencia 9 respondió 201 y generó avisos normales. 07-veneno-dlq.json conserva ambos conteos y la respuesta.

En la entrega ambos modos defectuosos están desactivados. No se cambian argumentos de una cola ya creada: para comparar configuraciones incompatibles haría falta migrarla o recrearla con el alcance correcto. No fue necesario borrar la cola en esta prueba.

Preguntas de cierre y límites

1 · Si RabbitMQ está caído, ¿qué pasa en esta versión?

El estado monetario puede quedar COMPLETADA mientras la publicación falla o sigue esperando. En la prueba, la transferencia 10 devolvió 504 por el gateway, aunque PostgreSQL ya decía COMPLETADA y los saldos eran 979 / 521. Al reactivar el broker el aviso sí llegó, mientras seguía viva la petición original; no afirmo que se perdiera en esa corrida. Si el proceso muriera entre el commit y publicar, no hay un relay persistente que lo recupere. Publisher confirm no vuelve atómicas esas dos escrituras. El experimento sin binding también mostró el error después del commit.

2 · ¿Por qué ack después de guardar?

Ack antes libera al broker: si luego muere el proceso, el aviso puede perderse. Guardar primero conserva la posibilidad de otra entrega si se cae antes del ack, pero puede repetir el evento. Por eso la garantía del broker es al menos una vez y el efecto idempotente depende de la marca UNIQUE y los avisos en una misma transacción. No se promete entrega exactamente una vez de extremo a extremo.

3 · Elección, razón y sacrificio

Elijo que el aviso sea asíncrono.
Porque Notificación puede estar apagada mientras el dinero se mueve y el broker conserva el recado.
Sacrifico la confirmación inmediata de que el cliente ya tiene el aviso; acepto atraso, duplicados que debo controlar y más operación. El broker sigue siendo una dependencia del publicador de esta versión.

Validación y procedencia

25 pruebas Java de Transferencia y 13 de Notificación aprobadas, además de los experimentos con RabbitMQ y PostgreSQL reales. Se comprueban confirms y returns, eventos, publicación tras el estado final, compensación, deduplicación, orden persistir/ack, JSON inválido y falla de almacenamiento. Los logs completos están en evidencia/tests-*.txt.

Se tomó como referencia la mensajería resuelta de la base oficial 06-poliglota-base y se adaptó sobre 05-mensajeria-base para esta recuperación, conservando avisos en PostgreSQL. Preparación con apoyo de Codex. No se implementó el reto opcional Outbox: queda la ventana de doble escritura. La base tampoco monta volumen de RabbitMQ; stop/start conserva el contenedor, down lo elimina.

Fuentes: guía S09 del curso (github.com/OscarRuiz21/sd-2027-1); rabbitmq.com/docs/confirms; microservices.io/patterns/data/transactional-outbox.html.